Análisis del problema

Desafío II: Amigos para siempre, con UdeABook

Informática II – Semestre 2026-2

1. Contexto y objetivo

Ude@Book es una red social orientada a conformar una comunidad virtual en el territorio colombiano. El desafío consiste en desarrollar únicamente la capa lógica de la plataforma, utilizando programación orientada a objetos en C++, de modo que se gestione de forma eficiente la información de usuarios, amigos, mensajes, solicitudes de amistad, álbumes y fotos. La interacción con el usuario se realiza por consola y los datos residen en almacenamiento permanente, del cual se cargan al iniciar y al cual se actualizan únicamente los cambios que el enunciado exige.

El enunciado enfatiza dos criterios de calidad: la eficiencia (en iteraciones y en memoria) y el buen diseño de las estructuras de datos propias, ya que no se permite el uso de contenedores de la STL ni de estructuras (`struct`). Por esta razón, gran parte del análisis se concentra en decidir qué estructura sirve mejor a cada operación solicitada.

2. Entidades del dominio y sus relaciones

A partir de la descripción del cliente se identifican las siguientes entidades.

- **Usuario.** Es la base del negocio. Se conoce su nombre de usuario (único), fecha de nacimiento, correo electrónico, ruta de la foto de perfil, ciudad, país y fecha de inscripción. Además posee amigos, mensajes enviados, mensajes recibidos, solicitudes de amistad recibidas y álbumes.
- **Álbum.** Tiene nombre, ruta de almacenamiento y una configuración de privacidad (público, privado o solo amigos). Pertenece a un único usuario y contiene cero o más fotos.
- **Foto.** Se conoce su fecha de captura, tamaño, ruta del archivo y conteo de likes. Pertenece a un único álbum.
- **Mensaje.** Tiene emisor, receptor, texto de hasta 512 caracteres, fecha de envío y un estado (leído o no leído).
- **Solicitud de amistad.** Relaciona a un usuario emisor con un usuario receptor, quien puede aceptarla.
- **Plataforma.** Coordina las demás entidades y expone las funcionalidades del menú.

Las relaciones de pertenencia son jerárquicas: la plataforma contiene usuarios, un usuario contiene álbumes y un álbum contiene fotos. La amistad es una relación simétrica de muchos a muchos entre usuarios: si A es amigo de B, entonces B es amigo de A. Esta simetría es la que justifica representarla mediante una matriz de adyacencia, como sugiere el enunciado.

3. Requisitos funcionales

El programa debe ofrecer las siguientes funcionalidades.

I. **Carga y actualización de datos.** Lectura y escritura desde y hacia el almacenamiento permanente. No aparece en el menú. El formato de los archivos debe diseñarse y explicarse en el informe.

II. **Ingreso a la plataforma y mejores amigos.** Inicio de sesión con credenciales y despliegue del menú principal junto con tres fotos de perfil de amigos elegidas al azar, con la opción de refrescarlas. La probabilidad de aparición depende de la cantidad de amigos en común.

III. **Administrar amistades.** Enviar solicitudes de amistad y aceptar las recibidas.

IV. **Visitar el perfil de un amigo.** Listar sus álbumes públicos o de amigos y recorrer las fotos de uno de ellos en modo carrusel, con las opciones salir, siguiente, previa y dar o quitar like.

V. **Ver sugerencias de amistad.** Dado un valor K entre 1 y 20, mostrar K sugerencias ordenadas por cantidad de amigos en común, con la opción de enviar solicitud.

VI. **Administrar mensajes.** Enviar mensajes a amigos, leer los mensajes nuevos y buscar mensajes recibidos por rango de fechas.

VII. **Medición del consumo de recursos.** Al terminar cada funcionalidad, mostrar la cantidad de iteraciones realizadas y la memoria total en uso en ese instante.

Además, el enunciado fija restricciones de lógica que deben respetarse: no enviar solicitudes a quienes ya son amigos ni a quienes ya enviaron una solicitud pendiente, no enviar mensajes vacíos ni a quienes no son amigos, y mostrar únicamente los álbumes que la configuración de privacidad permita ver. No se requiere registro, eliminación de cuentas, edición de álbumes y fotos, eliminación de amistades ni respuesta directa a mensajes.

4. Restricciones del desarrollo

- El lenguaje es C++ bajo el paradigma de POO, con al menos tres clases y programación modular en archivos de cabecera y de implementación.
- No se permiten contenedores de la STL, `struct` ni librerías de terceros que resuelvan asuntos esenciales. Las estructuras de datos deben ser propias y usar memoria dinámica. Solo se autoriza la librería para generar números aleatorios.
- El almacenamiento permanente no puede usarse como área de procesamiento o intercambio de datos.
- Se deben aplicar sobrecarga de al menos dos operadores, funciones amigas, constructores de copia y plantillas.
- Se debe preferir el paso por referencia frente a copias innecesarias de datos estructurados.
- El sistema debe poder demostrarse con al menos 1000 usuarios registrados.

5. Supuestos y aclaraciones pendientes

Durante el análisis se encontraron puntos que el enunciado no precisa. Se adoptaron los siguientes supuestos, que deben confirmarse con el cliente.

- **Credenciales.** El enunciado pide iniciar sesión con credenciales personales, pero no incluye una contraseña entre los datos del usuario. Se asume que cada usuario posee una contraseña almacenada junto a sus datos.
- **Prioridad de visualización.** Los rangos dados son 10 o menos, entre 11 y 20, y más de 21 amigos en común, de modo que el valor 21 queda sin cubrir. Se asume que el último rango comienza en 21. Se asume también que los amigos en común se cuentan entre el usuario que inició sesión y cada uno de sus amigos.
- **Likes.** Para poder quitar un like es necesario saber quiénes lo dieron. Se asume que cada foto conserva la lista de usuarios que le dieron like, además del conteo.
- **Orden de los mensajes.** Se asume que el orden por fecha de recepción coincide con el orden por fecha de envío.
- **Cadenas de texto.** Se debe confirmar si el tipo de cadena de la biblioteca estándar está permitido; de no estarlo, será necesario implementar una clase propia para manejar texto.

6. Consideraciones de eficiencia y estructuras de datos

Dado que la eficiencia es parte central de la evaluación, se analizó la operación dominante de cada funcionalidad y se eligió la estructura que la hace más barata.

6.1. Usuarios

Las búsquedas de usuarios por nombre son frecuentes: enviar solicitudes, visitar perfiles y enviar mensajes. Como en esta versión no hay registros ni eliminaciones, el conjunto de usuarios es estable, por lo que resulta conveniente un arreglo dinámico ordenado por nombre de usuario. Esto permite búsqueda binaria con costo logarítmico y hace que la posición de cada usuario sirva como identificador estable.

6.2. Amistades

La relación de amistad se representa con una matriz de adyacencia en la que cada fila es un arreglo de bits. Con 1000 usuarios, la matriz ocupa aproximadamente 125 KB. Consultar si dos usuarios son amigos cuesta tiempo constante. Contar los amigos en común entre dos usuarios se reduce a combinar sus filas con la operación lógica Y y contar los bits en uno, procesando varios usuarios por cada palabra de memoria. La principal desventaja es que la memoria crece con el cuadrado del número de usuarios, lo cual es aceptable para el tamaño de la demostración, pero debe señalarse como limitación.

6.3. Mensajes y solicitudes

Los mensajes de cada usuario se almacenan en arreglos dinámicos que crecen en orden cronológico, de modo que permanecen ordenados por fecha sin costo adicional. Esto permite buscar un rango

de fechas con dos búsquedas binarias, una para cada extremo, y recorrer después solo los mensajes del rango. Como al leer los mensajes nuevos se muestran todos y luego se marcan como leídos, los mensajes no leídos siempre forman el tramo final de la lista, por lo que basta conservar la posición del primero de ellos. Las solicitudes pendientes se guardan en el usuario receptor, lo que permite verificar duplicados revisando solo esa lista.

6.4. Álbumes y fotos

Cada usuario guarda sus álbumes y cada álbum sus fotos en arreglos dinámicos. El carrusel se implementa con un índice que avanza y retrocede de forma circular mediante el resto de la división, sin copiar las fotos. La visibilidad de un álbum se decide con la configuración de privacidad y, en el caso de álbumes para amigos, con una consulta de tiempo constante a la matriz de amistades.

6.5. Contenedor genérico

Los arreglos dinámicos se implementan una sola vez mediante una plantilla propia que gestiona la capacidad, el crecimiento y la liberación de memoria. De este modo se evita duplicar código para cada tipo de elemento y se cumple el requisito de usar plantillas.

7. Alternativa de solución propuesta

7.1. Mejores amigos con probabilidad ponderada

A cada amigo del usuario se le asigna un peso según la cantidad de amigos en común: uno, dos y medio, o tres y medio. Para trabajar con enteros se duplican los pesos. Luego se elige un amigo con el método de la ruleta: se genera un número aleatorio menor que la suma de los pesos y se recorre la lista acumulando pesos hasta superarlo. El amigo elegido se retira del sorteo y el procedimiento se repite hasta completar tres, lo que evita fotos repetidas en un mismo grupo. Al actualizar el grupo se excluyen los amigos mostrados anteriormente hasta agotar a todos, momento en que se reinicia el registro. Con ello se evita también un patrón fijo de visualización.

7.2. Sugerencias de amistad

Para el usuario que consulta, se recorre cada uno de sus amigos y, para cada amigo de ese amigo que no sea el propio usuario ni ya un amigo suyo, se incrementa un contador asociado. Al terminar, cada contador indica la cantidad de amigos en común del candidato. Como K es a lo sumo 20, se seleccionan los K mayores mediante K pasadas sobre los contadores, sin necesidad de ordenar la totalidad de los candidatos. El costo es proporcional a la suma de los grados de los amigos del usuario.

7.3. Administración de amistades

Al enviar una solicitud se valida que el destinatario exista, que no sea el propio usuario, que no sean ya amigos y que no exista una solicitud pendiente en cualquiera de los dos sentidos. Al aceptarla, se registra la amistad en la matriz en ambos sentidos y se retira la solicitud de la lista de pendientes.

7.4. Mensajería

Al enviar un mensaje se valida que el texto no esté vacío, que no supere los 512 caracteres y que el destinatario sea amigo. El mensaje se agrega a la lista de enviados del emisor y se referencia desde la lista de recibidos del receptor, evitando duplicar su contenido en memoria.

8. Medición del consumo de recursos

Para contar las iteraciones se utiliza un contador compartido que se incrementa en cada ciclo de las funciones implementadas, incluidos los ciclos internos de la plantilla de listas y de la matriz de amistades, de modo que se contabilicen tanto las iteraciones directas como las indirectas. Cuando una funcionalidad invoca un componente externo, como el generador de números aleatorios, se reporta su uso.

Para medir la memoria, cada clase ofrece una operación que devuelve el espacio que ocupa: el tamaño del propio objeto, más los bloques de memoria dinámica que posee, más lo que ocupan los objetos que contiene. La plataforma suma estos valores para todas las estructuras existentes y a ello se agrega el espacio de las variables locales y los parámetros por valor involucrados en la funcionalidad ejecutada. Ambas métricas se muestran en pantalla al finalizar cada funcionalidad.